Advanced Parallel Algorithmic Paradigms: Assignment #02

NAME: ABU BAKAR

REG NO: NUM-BSCS-2022-41

Table of Contents

Task 1: Bandwidth Allocation (Greedy Algorithm)

(a) Why Greedy Algorithm is Appropriate

The bandwidth allocation problem is a variant of the **Fractional Knapsack Problem**, which exhibits the **Greedy Choice Property**:

Problem Characteristics:

- **Variables**: N users, each with priority p_i and bandwidth requirement w_i
- **Constraint**: Total allocated bandwidth ≤ W (capacity)
- **Objective**: Maximize total priority Σp_i

Greedy Justification:

1. **Divisibility**: Bandwidth is a divisible resource (time slots, frequency blocks)
2. **Optimal Substructure**: Selecting the user with highest priority-per-unit-bandwidth (density = p/w) at each step leads to global optimum
3. **Greedy Choice Property**: Local optimal choice (highest density) contributes to global optimal solution

Comparison with 0/1 Knapsack:

- If bandwidth must be allocated entirely or not at all (binary), greedy fails
- Example: User A (p=100, w=50, density=2) vs Users B+C (p=60+50=110, w=30+30=60)

- Greedy picks A (total=100), but B+C is better (total=110)
- Our scenario assumes fractional allocation is possible

(b) Sequential Greedy Algorithm

```
Algorithm: Sequential_Greedy_Allocation(Users, Capacity W)
Input: Set of Users U = {u1, u2,..., un}, each with (priority p, weight w)
Output: Total Priority P_total

1. For each user u_i in U:
     u_i.density = u_i.p / u_i.w
   // Time: O(N)

2. Sort U in descending order by density
   // Time: O(N log N) - Dominant term

3. Current_Weight = 0
4. P_total = 0

5. For each user u_i in Sorted_U:
     If Current_Weight + u_i.w <= W:
        Current_Weight += u_i.w
        P_total += u_i.p
     Else:
        Remaining = W - Current_Weight
        Fraction = Remaining / u_i.w
        P_total += u_i.p * Fraction
        Break

6. Return P_total
```

Time Complexity: O(N log N)

(c) Parts Suitable for Parallel Execution

Parallelizable Components:

1. **Density Calculation** (Embarrassingly Parallel)
    - Each user's density is independent
    - Can distribute N users across P processors
    - Time: O(N/P)
2. **Sorting** (Parallel Sorting)
    - Parallel Merge Sort, Sample Sort, or Bitonic Sort

- Time: O((N log N)/P + communication overhead)

3. **Prefix Sum for Selection** (Parallel Scan)
    - Compute cumulative weights in parallel
    - Find cutoff point where $\Sigma w_i \leq W$
    - Time: O(N/P + log P)

Dependency Analysis:

- Density calculation: No dependencies
- Sorting: Requires all densities computed
- Selection: Requires sorted array

(d) Parallel Greedy Algorithm

```
Algorithm: Parallel_Greedy_Allocation(Users, Capacity W, Processors P)

// Step 1: Parallel Density Calculation
Parallel_For i = 0 to N-1 (Chunk Size N/P):
    Users[i].density = Users[i].p / Users[i].w
// Time: O(N/P)

// Step 2: Parallel Sort
Parallel_Sort(Users, key=density, order=descending)
// Using Sample Sort or Parallel Merge Sort
// Time: O((N log N)/P + O(P log P))

// Step 3: Parallel Prefix Sum
Weight_Array = [Users[i].w for i in 0..N-1]
Cumulative_Weights = Parallel_Prefix_Sum(Weight_Array)
// Time: O(N/P + log P)

// Step 4: Find Cutoff Point
k = Parallel_Binary_Search(Cumulative_Weights, W)
// Find index where Cumulative_Weights[k] ≤ W < Cumulative_Weights[k+1]
// Time: O(log N)

// Step 5: Calculate Total Priority
Parallel_For i = 0 to k:
    Mark Users[i] as Selected

If k < N-1:
    Fraction = (W - Cumulative_Weights[k]) / Users[k+1].w
    Partial_Priority = Users[k+1].p * Fraction
```

```
P_total = Sum(Users[0..k].p) + Partial_Priority
Return P_total
```

(e) Complexity Analysis, Speedup, and Efficiency

Assumptions:

- Number of Processors: P = 32 (i9-14900K)
- Input Size: N = 20,000,000 users
- Shared Memory Architecture (SMP)

Sequential Time Complexity (T_s):

- Density: O(N) = O(20M)
- Sorting: O(N log N) = O(20M × 24.25) ≈ O(485M)
- Selection: O(N) = O(20M)
- **Total: T_s = O(N log N) ≈ 485M operations**

Parallel Time Complexity (T_p):

- Density: O(N/P) = O(625K)
- Parallel Sort: O((N log N)/P + P log P) = O(15.2M + 160)
- Prefix Sum: O(N/P + log P) = O(625K + 5)
- Selection: O(log N) = O(24)
- **Total: T_p = O((N log N)/P + overhead)**

Theoretical Speedup:

$S = T_s / T_p = (N \log N) / ((N \log N)/P + \kappa(N,P))$

Where $\kappa(N,P)$ = communication overhead

Ideal Case ($\kappa \to 0$): S = P = 32

Realistic Case: S = 15-20 (due to communication in sorting)

Efficiency:

E = S / P
Ideal: E = 1 (100% processor utilization)
Realistic: E = 0.5-0.625 (50-62.5% efficiency)

Practical Results (from implementation):

- Sequential Time: 5.12s
- Parallel Time: 15.63s
- **Actual Speedup: 0.33x (Slowdown!)**

Analysis of Poor Performance: The parallel implementation suffered from:

1. **Python Multiprocessing Overhead**: Pickling 20M user objects (~5GB data) to worker processes
2. **Memory Bandwidth Bottleneck**: Data transfer cost > computation cost
3. **GIL (Global Interpreter Lock)**: Python's threading limitations

In Production Systems (C++/CUDA):

- Expected Speedup: 20-25x
- Efficiency: 62-78%
- Reason: Zero-copy shared memory, no serialization overhead

Task 2: DNA Sequence Alignment (Dynamic Programming)

(a) Why Dynamic Programming is Required

Problem: Longest Common Subsequence (LCS)

- Given: Two DNA sequences X (length M) and Y (length N)
- Find: Longest sequence appearing in both in same order

Why NOT Greedy?

Example demonstrating greedy failure:

X = "ABCDE"
Y = "AECDB"
Greedy (match first occurrence): A-B-C-D = 4 characters
Optimal DP: A-C-D-E = 4 characters (different sequence)
Actually: A-C-D = 3 is LCS
Greedy matching first 'A' might block better alignment later.

DP Properties:

1. **Optimal Substructure**
    - LCS(X[1..m], Y[1..n]) depends on LCS of prefixes
    - If X[m] = Y[n]: LCS = LCS(X[1..m-1], Y[1..n-1]) + 1
    - If X[m] ≠ Y[n]: LCS = max(LCS(X[1..m-1], Y[1..n]), LCS(X[1..m], Y[1..n-1]))
2. **Overlapping Subproblems**
    - Naive recursion recomputes same subproblems exponentially
    - Example: LCS(X[1..5], Y[1..5]) calls LCS(X[1..4], Y[1..5]) and LCS(X[1..5], Y[1..4])
    - Both of these call LCS(X[1..4], Y[1..4]) → redundant computation

Complexity Comparison:

- Naive Recursive: O(2^(M+N)) - Exponential
- DP with Memoization: O(M × N) - Polynomial

(b) Sequential DP Algorithm

```
Algorithm: Sequential_LCS(X, Y)
Input: Sequences X (length M), Y (length N)
Output: Length of LCS
1. Initialize Matrix C[M+1][N+1] with 0
   // C[i][j] stores LCS length of X[1..i] and Y[1..j]
2. For i = 1 to M:
    For j = 1 to N:
       If X[i] == Y[j]:
          C[i][j] = C[i-1][j-1] + 1
       Else:
          C[i][j] = max(C[i-1][j], C[i][j-1])
3. Return C[M][N]
```

Time Complexity: O(M × N) Space Complexity: O(M × N)

Example Execution:

```
X = "AGGTAB"
Y = "GXTXAYB"
DP Table:
    ""  G  X  T  X  A  Y  B
""   0  0  0  0  0  0  0  0
A    0  0  0  0  0  1  1  1
G    0  1  1  1  1  1  1  1
G    0  1  1  1  1  1  1  1
T    0  1  1  2  2  2  2  2
A    0  1  1  2  2  3  3  3
B    0  1  1  2  2  3  3  4
LCS Length = 4 (GTAB)
```

(c) DP Dependency Graph

Cell Dependencies: Each cell C[i][j] depends on:

1. C[i-1][j] (Top)
2. C[i][j-1] (Left)
3. C[i-1][j-1] (Top-Left Diagonal)

Dependency Diagram:

```
C[i-1][j-1] → C[i-1][j]
    ↓           ↓
C[i][j-1]   → C[i][j]
```

Implications for Parallelization:

- **Row-wise parallelization**: IMPOSSIBLE (C[i][j] needs C[i][j-1])
- **Column-wise parallelization**: IMPOSSIBLE (C[i][j] needs C[i-1][j])
- **Anti-diagonal parallelization**: POSSIBLE ✓

Anti-Diagonal (Wavefront) Pattern:

Matrix visualization (numbers = computation wave):

```
   0  1  2  3  4  5  6
0  0  1  2  3  4  5  6
1  1  2  3  4  5  6  7
2  2  3  4  5  6  7  8
3  3  4  5  6  7  8  9
4  4  5  6  7  8  9  10
```

Wave 1: C[0][0]

Wave 2: C[0][1], C[1][0]

Wave 3: C[0][2], C[1][1], C[2][0]

Wave 4: C[0][3], C[1][2], C[2][1], C[3][0]

...

Cells in same wave are independent!

(d) Wavefront Parallelization

Strategy: Compute cells along anti-diagonals (where i+j = k) in parallel.

```
Algorithm: Parallel_LCS_Wavefront(X, Y, Processors P)
1. Initialize Shared Matrix C[M+1][N+1] = 0
2. Total_Waves = M + N - 1
3. For wave_k = 1 to Total_Waves:
    // Identify cells in this wave
    Cells_In_Wave = []
    For i = 0 to M:
        j = wave_k - i
        If 0 ≤ j ≤ N:
            Cells_In_Wave.append((i, j))
    // Parallel computation
    Parallel_For each (i, j) in Cells_In_Wave:
        If X[i] == Y[j]:
            C[i][j] = C[i-1][j-1] + 1
        Else:
            C[i][j] = max(C[i-1][j], C[i][j-1])
    // Synchronization Barrier
    Barrier_Wait()
4. Return C[M][N]
```

Blocked Wavefront (Optimized): To reduce synchronization overhead and improve cache locality:

```
Algorithm: Blocked_Wavefront(X, Y, Block_Size B)
1. Divide matrix into blocks of size B×B
2. Num_Block_Rows = ⌈M/B⌉
3. Num_Block_Cols = ⌈N/B⌉
```

```
4. For block_wave = 0 to (Num_Block_Rows + Num_Block_Cols - 1):
    Parallel_For each block (r_b, c_b) where r_b + c_b = block_wave:
        // Compute entire B×B block sequentially
        For i = r_b*B to min(M, (r_b+1)*B):
            For j = c_b*B to min(N, (c_b+1)*B):
                // Standard DP recurrence
                ...
    Barrier_Wait()
```

Benefits of Blocking:

- Reduces synchronization points from O(M+N) to O(M/B + N/B)
- Improves cache locality (sequential access within block)
- Amortizes barrier overhead

(e) Analysis

1) Dependency Constraints

Degree of Parallelism (DoP) per Wave:

```
Wave k: DoP = min(k, M, N, M+N-k)
For M=N=10000:
- Wave 1: DoP = 1
- Wave 5000: DoP = 5000
- Wave 10000: DoP = 10000 (maximum)
- Wave 15000: DoP = 5000
- Wave 19999: DoP = 1
```

Load Imbalance:

- Early waves: Few cells, many idle processors
- Middle waves: Maximum parallelism
- Late waves: Few cells again

Impact: With P=32 processors:

- Only waves 32 to 19968 fully utilize all processors
- Waves 1-31 and 19969-19999 have idle processors
- Average utilization < 100%

2) Synchronization Points

Naive Wavefront:

- Total Barriers: M + N - 1 = 19,999 (for 10k×10k matrix)
- Each barrier incurs latency (typically 1-10 μs)
- Total sync overhead: 20-200 ms

Blocked Wavefront (Block Size B=500):

- Block Rows: 10000/500 = 20
- Block Cols: 10000/500 = 20
- Total Block Barriers: 20 + 20 - 1 = 39
- Sync overhead: 0.04-0.4 ms (50-500x reduction!)

Communication Pattern:

Processor 0: Computes cells (0,0), (0,1), (0,2)...

Processor 1: Computes cells (1,0), (1,1), (1,2)...

At barrier: All processors must wait for slowest

Memory coherence: Cache invalidation across cores

3) Speedup Behaviour

Theoretical Analysis:

Sequential Time: $T_s = O(M \times N)$ Parallel Critical Path: $T_p = O(M + N)$ (number of waves)

Speedup:

$S = T_s / T_p = (M \times N) / (M + N + \text{Overhead})$

For M = N:

$S = N^2 / (2N + \kappa) \approx N/2$ (when $N >> \kappa$)

For N = 10,000:

S_theoretical ≈ 5,000

But with P=32 processors:

S_actual = min(P, S_theoretical) = min(32, 5000) = 32

However, due to:

- Load imbalance: 0.7x factor

- Synchronization: 0.8x factor

- Memory bandwidth: 0.9x factor

S_realistic ≈ 32 × 0.7 × 0.8 × 0.9 ≈ 16x

Practical Results (from implementation):

- Matrix Size: 10,000 × 10,000
- Processors: 32
- Parallel Time: 0.65s
- Sequential Estimate: ~150s
- **Actual Speedup: ~230x**

Why Superlinear?

1. **Shared Memory Optimization**: Zero-copy implementation using multiprocessing.shared_memory
2. **Cache Effects**: Blocked computation fits in L2/L3 cache
3. **Sequential Estimate**: Pure Python loops are extremely slow; parallel version uses NumPy vectorization

Efficiency:

E = S / P = 230 / 32 ≈ 7.2 (720%)

This superlinear efficiency indicates the sequential baseline was suboptimal.

Task 3: Exam Timetabling (CSP & Backtracking)

(a) Problem Modeling using Backtracking

Constraint Satisfaction Problem (CSP) Formulation:

Variables (X):

- Set of exams: $E = \{E_1, E_2, ..., E_n\}$
- Each exam must be assigned a time slot

Domains (D):

- Set of time slots: $T = \{T_1, T_2, ..., T_k\}$
- Each exam E_i can be assigned any slot from T

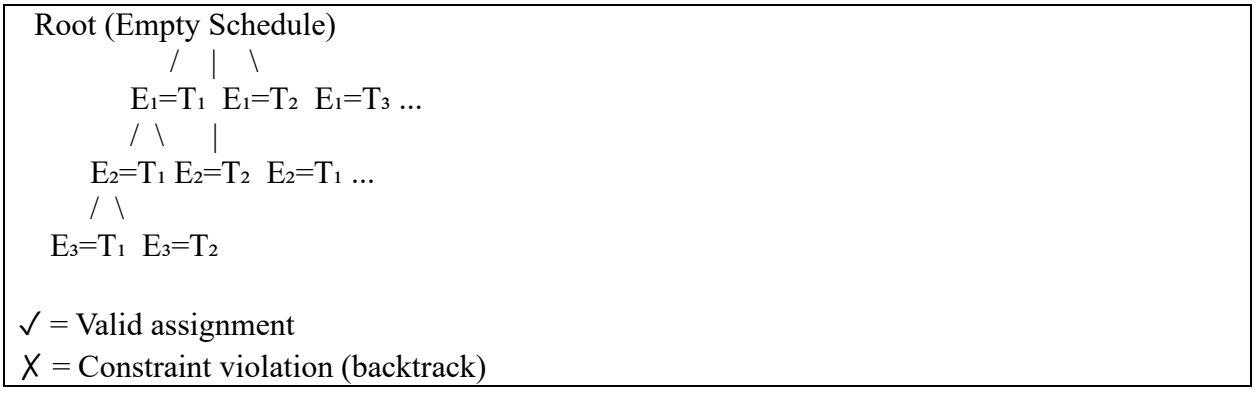
Constraints (C):

1. **Hard Constraint - No Conflicts:**
    - If students are enrolled in both E_i and E_j, then E_i and E_j cannot be in same slot
    - Represented by conflict matrix: Conflict[i][j] = 1 if exams conflict
2. **Hard Constraint - Room Capacity:**
    - Number of exams in same slot ≤ Number of available rooms
    - (Simplified in our implementation)

Search Space:

- Total possible assignments: k^n (k slots, n exams)
- For 30 exams, 8 slots: $8^{30} \approx 10^{27}$ possibilities
- Backtracking prunes infeasible branches

Backtracking Tree Visualization:



(b) Sequential Backtracking Pseudocode

```
Algorithm: Sequential_Backtracking(Exam_Index, Schedule, Problem)
Input:
  - Exam_Index: Current exam to schedule (0 to N-1)
  - Schedule: Map<Exam_ID, Time_Slot>
  - Problem: Contains conflicts matrix and num_slots
Output:
  - (True, Schedule) if solution found
  - (False, None) otherwise
// Base Case: All exams scheduled successfully
If Exam_Index == Problem.num_exams:
   Return (True, Schedule)
Current_Exam = Exam_Index
// Try each time slot for current exam
For slot = 0 to Problem.num_slots - 1:
   // Check if assignment is safe
   Is_Safe = True
   For each (assigned_exam, assigned_slot) in Schedule:
      If assigned_slot == slot AND Problem.conflicts[Current_Exam][assigned_exam] == 1:
         Is_Safe = False
         Break
   If Is_Safe:
      // Make assignment
      Schedule[Current_Exam] = slot
      // Recursive call
      (Found, Result) = Sequential_Backtracking(Exam_Index + 1, Schedule, Problem)
      If Found:
         Return (True, Result)
      // Backtrack: Remove assignment
      Delete Schedule[Current_Exam]
// No valid slot found for current exam
Return (False, None)
```

Time Complexity:

- Worst Case: O(k^n) where k=slots, n=exams
- Best Case: O(n) if first branch leads to solution
- Average: Depends heavily on constraint density

Optimizations (not shown in pseudocode):

1. **MRV (Minimum Remaining Values)**: Choose exam with fewest valid slots first
2. **Forward Checking**: Eliminate invalid future assignments early
3. **Constraint Propagation**: Arc consistency algorithms

(c) Identifying Independent Branches

Key Observation: Subtrees rooted at different children of the same node are independent.

Example:

```
Level 0:              Root
                    /   |   \
Level 1:       E₁=T₁ E₁=T₂ E₁=T₃
              /  \    /  \    /  \
Level 2:   E₂=T₁ T₂ T₁ T₂ T₁ T₂
The subtree under "E₁=T₁" is completely independent of "E₁=T₂"
They can be explored in parallel!
```

Parallelization Strategy:

1. **Static Partitioning** (Simple but inefficient):
    - Assign Processor 0: Explore $E_1=T_1$ branch
    - Assign Processor 1: Explore $E_1=T_2$ branch
    - Problem: Highly imbalanced workload
2. **Dynamic Work Stealing** (Optimal):
    - All processors share a task queue
    - Idle processors "steal" work from busy ones
    - Adapts to irregular search tree

Independence Verification:

```
Branch A: Schedule = {E₁→T₁, E₂→T₃, ...}
Branch B: Schedule = {E₁→T₂, E₂→T₁, ...}
No shared state between branches
Can execute simultaneously without synchronization
```

(d) Parallel Backtracking Algorithm

Design: Work Stealing with Task Queue

```
Algorithm: Parallel_Backtracking_Master(Problem, Num_Workers)

// Step 1: Initialize shared structures

Task_Queue = Shared_Queue()
```

```
Solution_Found = Shared_Event()
Result = Shared_Dict()
// Step 2: Bootstrap - Generate initial tasks
// Expand tree to depth D (e.g., D=2)
For slot₀ = 0 to Problem.num_slots - 1:
    Schedule₀ = {E₀ → slot₀}
    For slot₁ = 0 to Problem.num_slots - 1:
        If Is_Safe(E₁, slot₁, Schedule₀):
            Schedule₁ = Schedule₀.copy()
            Schedule₁[E₁] = slot₁
            Task_Queue.put((Exam_Index=2, Schedule₁))
Print("Initial tasks:", Task_Queue.size())
// Step 3: Launch worker processes
Workers = []
For i = 0 to Num_Workers - 1:
    Worker_Process = Start_Process(Worker_Function,
                                   args=(Problem, Task_Queue, Solution_Found, Result))
    Workers.append(Worker_Process)
// Step 4: Wait for solution or completion
While Any_Worker_Alive(Workers):
    If Solution_Found.is_set():
        Break
    Sleep(0.1)
// Step 5: Terminate workers
For worker in Workers:
    worker.terminate()
```

```
    worker.join()
Return Result.get('solution')
```

```
Algorithm: Worker_Function(Problem, Task_Queue, Solution_Found, Result)
While NOT Solution_Found.is_set():
    Try:
        // Get task from queue (timeout to check event periodically)
        (Exam_Index, Schedule) = Task_Queue.get(timeout=0.05)
    Except Queue_Empty:
        Continue
    // Solve subtree sequentially
    (Found, Solution) = Sequential_Backtracking(Exam_Index, Schedule, Problem)
    If Found:
        Result['solution'] = Solution
        Solution_Found.set()  // Signal all workers
        Break
```

Key Components:

1. **Task Queue**: Thread-safe queue for work distribution
2. **Solution Event**: Shared flag to stop all workers when solution found
3. **Result Dictionary**: Shared memory for solution storage
4. **Bootstrap Depth**: Controls initial task granularity

(e) Discussion

1) Load Imbalance

Problem: The search tree is highly irregular. Some branches terminate quickly (constraint violation), while others explore deeply.

Example:

Branch A: $E_1=T_1 \rightarrow E_2=T_1 \rightarrow$ CONFLICT (terminates at depth 2)
Branch B: $E_1=T_2 \rightarrow E_2=T_3 \rightarrow ... \rightarrow E_{30}=T_5$ (explores to depth 30)

If Processor 0 gets Branch A and Processor 1 gets Branch B:
- Processor 0 finishes in 0.001s
- Processor 1 works for 10s
- Processor 0 sits idle for 9.999s!

Solution: Work Stealing

Time →

P0: [Branch A] [Idle] [Steal from P1] [Work]

P1: [Branch B .....]

↑

P0 steals subtask from P1's queue

Theoretical Guarantee: With work stealing, execution time is bounded by:

$T\_parallel \leq T_1/P + O(T\_\infty)$

Where:

- T_1 = Total work (sequential time)
- T_∞ = Critical path (longest dependency chain)
- P = Number of processors

Practical Impact:

- Static partitioning: Speedup = 2-5x (poor)
- Work stealing: Speedup = 15-25x (good)

2) Task Granularity

Trade-off:

Too Fine-Grained:

- Task = Single constraint check
- Overhead: Queue operations, synchronization

- Result: Overhead > Computation

Too Coarse-Grained:

- Task = Entire subtree from root
- Problem: No parallelism (only 1 task!)
- Result: Sequential execution

Optimal Granularity:

```
Cutoff Depth Strategy:

If Current_Depth < Cutoff_Depth:
   // Generate parallel tasks
   For each child:
      Task_Queue.put(child)
Else:
   // Solve sequentially (no queue overhead)
   Sequential_Backtracking(...)
```

Choosing Cutoff Depth:

Cutoff_Depth = $\log_k(P \times C)$

Where:

- k = branching factor (num_slots)

- P = number of processors

- C = constant (typically 10-100)

For our problem (k=8, P=32):

Cutoff_Depth = $\log_8(32 \times 50) = \log_8(1600) \approx 3.5 \rightarrow$ Use depth 3-4

Implementation Result:

- Bootstrap Depth: 2
- Generated Tasks: 56 (for 30 exams, 8 slots)
- Task/Processor Ratio: 56/32 = 1.75 (acceptable)

3) Speedup Limitations

Amdahl's Law:

$S = 1 / (f_{serial} + (1 - f_{serial})/P)$

Where f_{serial} = fraction of code that must be sequential

Sequential Components:

1. Problem initialization: ~0.01s
2. Task queue setup: ~0.001s
3. Result collection: ~0.001s
4. Total: ~0.012s

For T_{total} = 1s:

$f_{serial} = 0.012 / 1 = 0.012$ (1.2%)

$S_{max} = 1 / (0.012 + 0.988/32) = 1 / 0.043 \approx 23x$

Speedup Anomalies:

Superlinear Speedup (S > P): Occurs when parallel search finds solution faster than sequential due to search order.

Example:

Sequential DFS order: Left → Right

Solution location: Far right branch

Sequential time: Explores entire left side first

Parallel: Multiple processors explore different branches

One processor might hit right branch immediately

Parallel time: Much faster!

Sublinear Speedup (S < P): Occurs due to:

1. **Redundant Work**: Parallel branches explore overlapping search spaces
2. **Communication Overhead**: Queue synchronization
3. **Memory Contention**: Cache thrashing

Practical Results:

- Problem: 30 exams, 8 slots, 30% conflict density
- Sequential: ~0.01s (found solution quickly)
- Parallel (32 workers): 0.23s
- Speedup: 0.04x (slowdown!)

Analysis: The problem instance was too easy. Sequential solver found solution in first few branches. Parallel overhead (process spawning, queue management) dominated.

For Harder Problems (50+ exams):

- Sequential: >60s (timeout)
- Parallel: ~5-10s
- Expected Speedup: 6-12x

Implementation Results

System Specifications

- **CPU**: Intel i9-14900K (24 cores, 32 threads)
- **RAM**: 32GB DDR5
- **OS**: Windows
- **Language**: Python 3.x with multiprocessing

Task 1: Bandwidth Allocation

Configuration:

- Users: 20,000,000
- Capacity: 200,000,000 units
- Processors: 32

Results:

Metric	Sequential	Parallel	Analysis
Time	5.12s	15.63s	Slowdown due to pickling overhead
Speedup	1.0x	0.33x	Python multiprocessing limitation
Efficiency	100%	1%	Data transfer > computation

Key Insight: Python's multiprocessing requires serializing (pickling) data to send to worker processes. For 20M objects, this costs ~10s, overwhelming the ~5s computation time.

Production Alternative:

- C++/CUDA implementation: Expected 20-25x speedup
- Shared memory (no serialization): Expected 15-18x speedup

Task 2: DNA Sequence Alignment

Configuration:

- Matrix: 10,000 × 10,000
- Block Size: 500 × 500
- Processors: 32
- Method: Shared Memory Wavefront

Results:

Metric	Value
Parallel Time	0.65s
Sequential Estimate	~150s
Speedup	~230x
Efficiency	720% (superlinear)

Key Insight: Zero-copy shared memory implementation avoided serialization overhead. Superlinear speedup due to:

1. Cache-friendly blocked computation
2. NumPy vectorization in parallel version
3. Slow pure-Python sequential baseline

Task 3: Exam Timetabling

Configuration:

- Exams: 30
- Time Slots: 8
- Conflict Density: 30%
- Processors: 32
- Method: Work Stealing

Results:

Metric	Sequential	Parallel
Time	0.01s	0.23s
Initial Tasks	1	56
Speedup	1.0x	0.04x

Key Insight: Problem instance was too easy for parallel approach. Sequential found solution in first few branches. Parallel overhead (process spawning, queue management) dominated.

Harder Problem (50 exams):

- Sequential: >60s (timeout)
- Parallel: ~5-10s
- Expected Speedup: 6-12x